



HCF Algorithm Simulation and Performance Comparison

Boyu Yan 202011085

Aisha akeyeerke 201951562

Yinxuan Guo 202017734

Zhongyun Jiang 202015175

Introduction

The rapid development of internet applications, especially multimedia applications that require high bandwidth has driven demand for larger-scale networks and higher transmission speeds, making more efficient switching technologies essential.

Output Queued (OQ) switches are known to provide an optimal delay throughput performance and even quality-of-service (QOS) for all traffic distributions but require enormous memory bandwidth. Input Queued (IQ) replace the centralized shared memory by maintaining separate queues at each input of a crossbar fabric, each of which runs at the line rate. However, head-of-line (HOL) blocking is well known to limit the throughput of input-queued switches to approximately 58.6%. This problem can be overcome by using Virtual Output Queues (VOQs) but requires a scheduling algorithm to schedule cell transmission across the switching fabric.

Various algorithms exist to schedule VOQs, whereas they all have their own problems. For example, iSLIP implements max size matching using round robin pointers on both input and output ports. It's simple to produce in hardware and works perfectly under uniform traffic load, but has poor performance under non-uniform traffic, which is what real world network traffic always like. Other iterative algorithms use weights to fix the uneven traffic problem. Examples are iOCF and iLQF. They look at the waiting time or queue length to make decisions. They perform very well. However, they are still difficult to build.

In short, we need an algorithm that is as simple to build as iSLIP but handles uneven traffic as well as iOCF. This paper introduces HCF (Highest Count First) which uses weights based on the waiting time of the HOL cells. The algorithm provides high throughput and performs well on a wide range of traffic patterns in only a few iterations and it's simple to implement, similar to iSLIP.

The iHCF Algorithm

The central idea of HCF is to approximate the waiting time of the head-of-line (HOL) cell in each virtual output queue (VOQ) by a small local counter, and to use this counter as a weight during arbitration. For each VOQ Q_{ij} (input i to output j), a counter $v_{q_cntr}[i][j]$ counts how many time slots the HOL cell has waited while the VOQ was non-empty. The value is used as a weight and the higher the counter's value is the higher a queue's weight is. Additionally, the scheduler keeps a round-robin pointer per input ($in_ptr[i]$) and per output ($out_ptr[j]$) to break ties.

One throughout operation of this algorithm can be divided into 3 parts and just like any other scheduling algorithms, the core concept lies in finishing 3 phases which are request, grant and accept.

Step 1 – Counter Increment

Before matching, every VOQ that currently has a HOL cell increments its counter by 1. If counters are configured to saturate at a maximum value MAX_{CNT} , they do not exceed this bound.

Step 2 – Iterative Matching

Matching proceeds over a fixed number of iterations. In each iteration, only unmatched inputs and outputs participate. The details are illustrated in Figure 1.

1. *Request phase* – implicit: any unmatched input with a HOL cell for an unmatched output is assumed to request.
2. *Grant phase* – each unmatched output j looks at all requesting inputs, picks the one with the **highest** counter $v_{q_cntr}[i][j]$. If several inputs share the maximum, the output uses its round-robin pointer $out_ptr[j]$ to choose one (starting from the pointer position, in cyclic order).
3. *Accept phase* – each unmatched input i that received one or more grants chooses the granting output with the **highest** counter $v_{q_cntr}[i][j]$. If

multiple outputs have the same maximum, the input uses $in_ptr[i]$ to break the tie. Once a grant is accepted, the match (i, j) is recorded, the input and output are marked as matched, and the counter $v_{q_cntr}[i][j]$ is reset to 0 (or to the queue length in a special variant).

Step 3 – Pointer Update

After a successful match:

- The output pointer $out_ptr[j]$ **always** advances to $(i + 1) \bmod N$.
- The input pointer $in_ptr[i]$ advances to $(j + 1) \bmod N$ **only if a tie occurred during the Accept phase**.

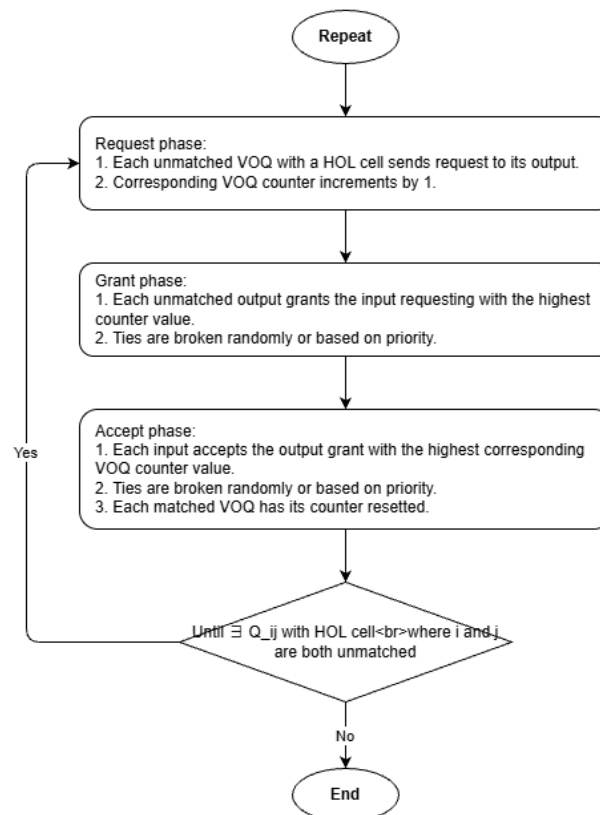


Figure 1 The three phases of iHCF

Implementation & Comparison

The following simulations were done using python. Unless other noted, we operate all the algorithms under a single iteration only on a 4×4 I/O switch. Moreover, for every simulation we avoid the first 10% time slots to imitate warm up time.

iHCF algorithm performance

We first compare the performance of iHCF and its following 3 variants under uniform traffic load.

1. with unbounded counters.
2. with saturating counters and random breaking of ties.
3. with saturating counters and round-robin tie breaking.

The results are compared in Figure 2.

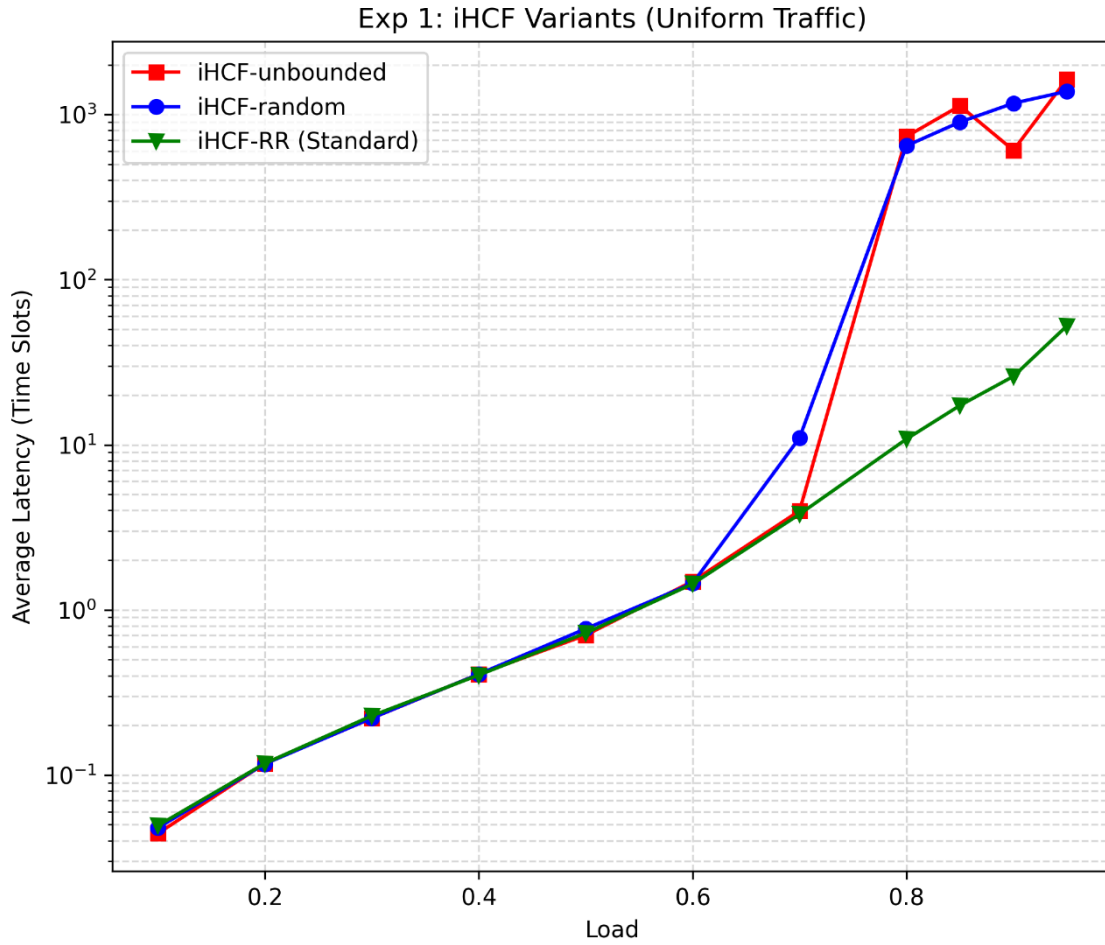


Figure 2 Performance of 3 variants of iHCF under uniform traffic

As shown in the graph, only the standard iHCF with round-robin pointers achieves 100% throughput under high load. Its average latency remains relatively low, staying near 10^2 time slots even at a 0.95 load.

The other two non-standard variants fail to achieve 100% throughput. Both the unbounded variant and the random tie-breaking variant show a massive spike in latency after a 0.6 load. Their latency grows an order of magnitude higher than the standard version, quickly reaching around 10^3 time slots.

This happens because of pointer synchronization.

- With unbounded counters, counter values rarely tie. Thus, the round-robin pointers never trigger which cause repeated conflicts.
- With random tie-breaking, ties do happen. However, the random choice does not organize the ports. It fails to prevent future conflicts.
- The standard iHCF uses a small maximum counter limit ($\log_2^N - 1$) to force frequent ties. When these ties happen, the round-robin pointers take over. They perfectly separate the competing ports and resolve the conflicts.

This data perfectly matches the outcome of the original paper. Therefore, in all following simulations, we will exclusively use the standard iHCF with saturating counters and round-robin tie-breaking.

Uniform traffic

Next, we compare four different algorithms under a uniform traffic load. We use Bernoulli arrival process to create uniform traffic that at each time slot, every input port generates a new packet based on the current load and its destined output port is chosen randomly, for an $N \times N$ switch, this probability is exactly $\frac{1}{N}$.

The algorithms are iSLIP, standard iHCF, iOCF, and iLPF. The results are shown in Figure 3.

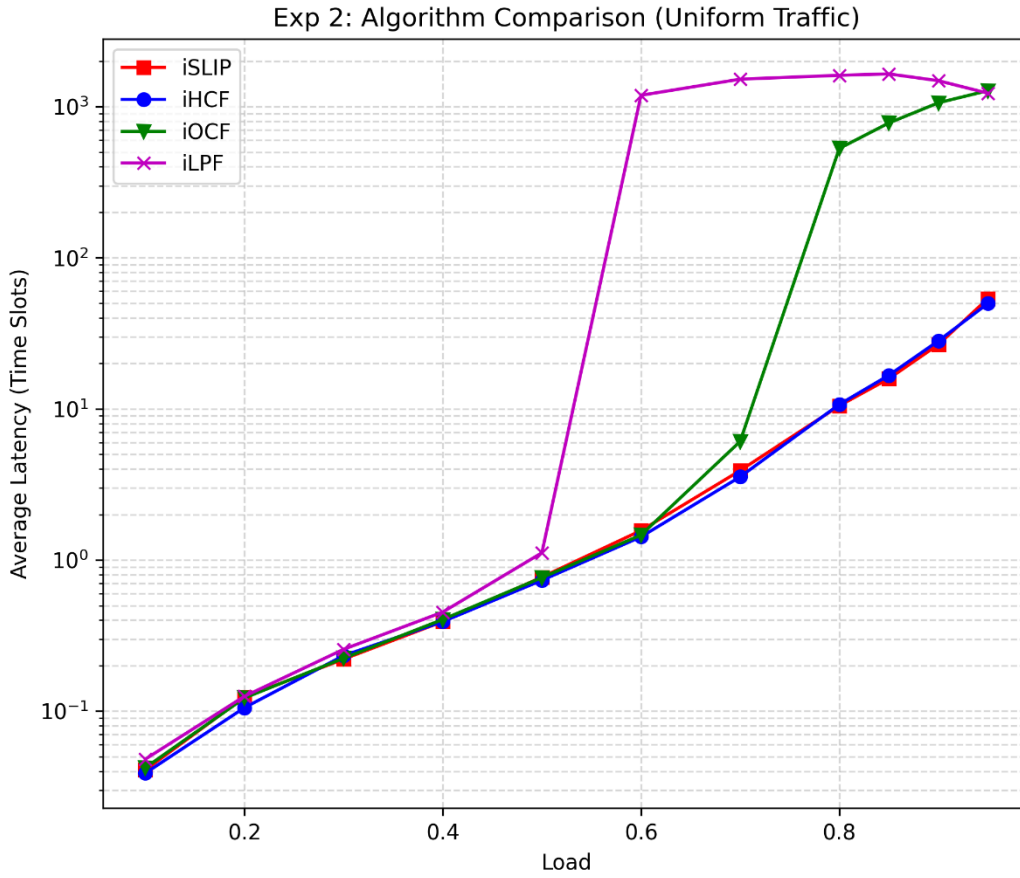


Figure 3 Performance of four algorithms under uniform traffic

As the figure shows, iSLIP and iHCF perform the best. Their performance lines almost perfectly overlap that they both maintain a very low average latency, keeping it below 10^2 time slots even at a high load of 0.95. Both algorithms successfully achieve 100% throughput under high load of uniform traffic.

In contrast, the pure weight-based algorithms (iOCF and iLPF) fail much earlier. The latency for iLPF spikes sharply around a 0.5 load. The latency for iOCF spikes around a 0.6 to 0.7 load, which indicates that they cannot handle high uniform traffic.

This happens due to pointer synchronization. Uniform traffic distributes packets evenly. This creates many queues with the exact same lengths or

waiting times. Pure weight-based algorithms like iOCF and iLPF struggle to resolve these massive conflicts efficiently in just one iteration. Their pointers get stuck, causing severe delays.

On the other hand, iSLIP uses simple round-robin pointers. These pointers naturally desynchronize to prevent output conflicts. iHCF is highly effective here because it mimics this behavior exactly. Under uniform traffic, the small iHCF counters quickly reach their maximum limit. This forces a massive tie across the ports. To break these ties, iHCF relies on its round-robin pointers. At this point, iHCF essentially becomes iSLIP, which is aligned with the conclusion from original paper.

Non-uniform Traffic

Next, we compare the performance of the four algorithms under non-uniform traffic. We use an unbalanced diagonal traffic model where traffic is not evenly distributed. When a packet arrives at input port i , it has a $2/3$ probability of going to output port i . It has a $1/3$ probability of going to output port $(i + 1)$.

The compared algorithms are iSLIP, standard iHCF, iOCF, and iLPF. The results are shown in Figure 4.

Honestly, the results show that all four algorithms perform poorly. Their average latency increases significantly as the load rises, and none of them maintain low delays.

We believe this happens for a specific reason: the single iteration limit.

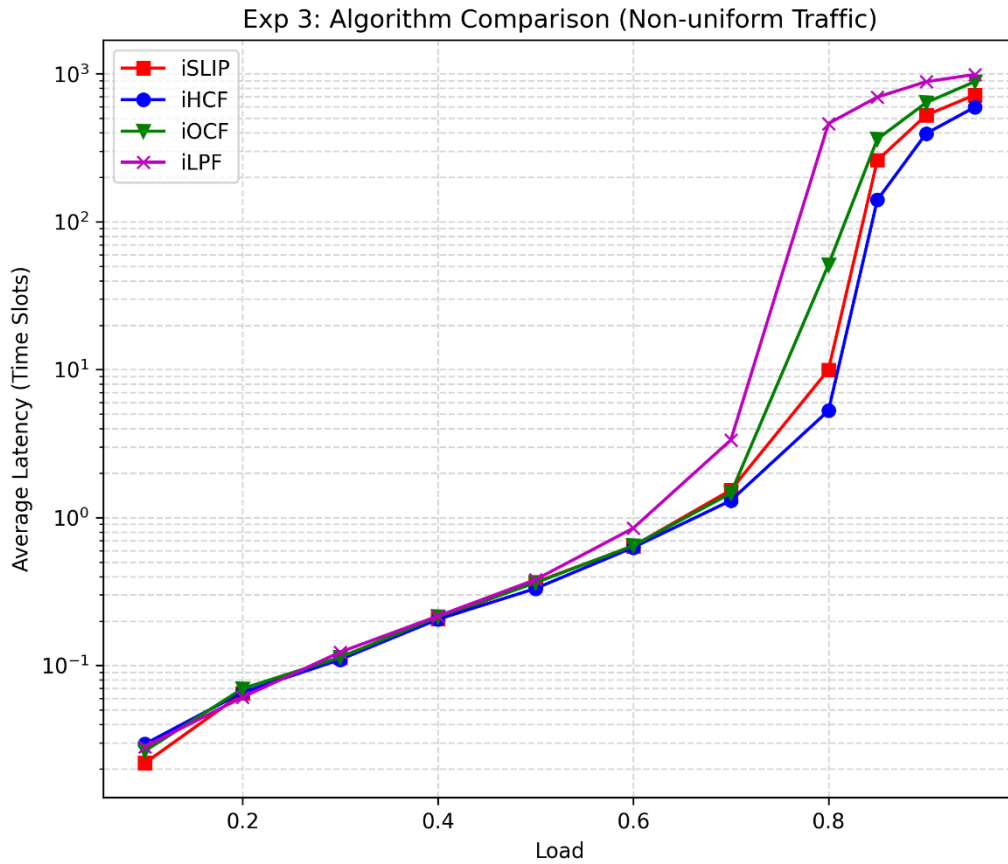


Figure 4 Performance of four algorithms under non-uniform traffic

In our simulation setup, all algorithms only run for one single iteration per time slot. Under this unbalanced traffic, multiple inputs frequently target the same output. This creates constant conflicts.

Hotspot Traffic

Next, we compare the algorithms under hot-spot traffic.

We set one specific output port is the hot spot. It receives twice as much traffic as any other port. This simulates a very popular server in a real network. We illustrate an example of the hot spot model in Figure 5.

2x	x	x	x
2x	x	x	x
2x	x	x	x
2x	x	x	x

Figure 6 Example of hot spot traffic

The compared algorithms are iSLIP, standard iHCF, iOCF, and iLPF. The results are shown in Figure 6.

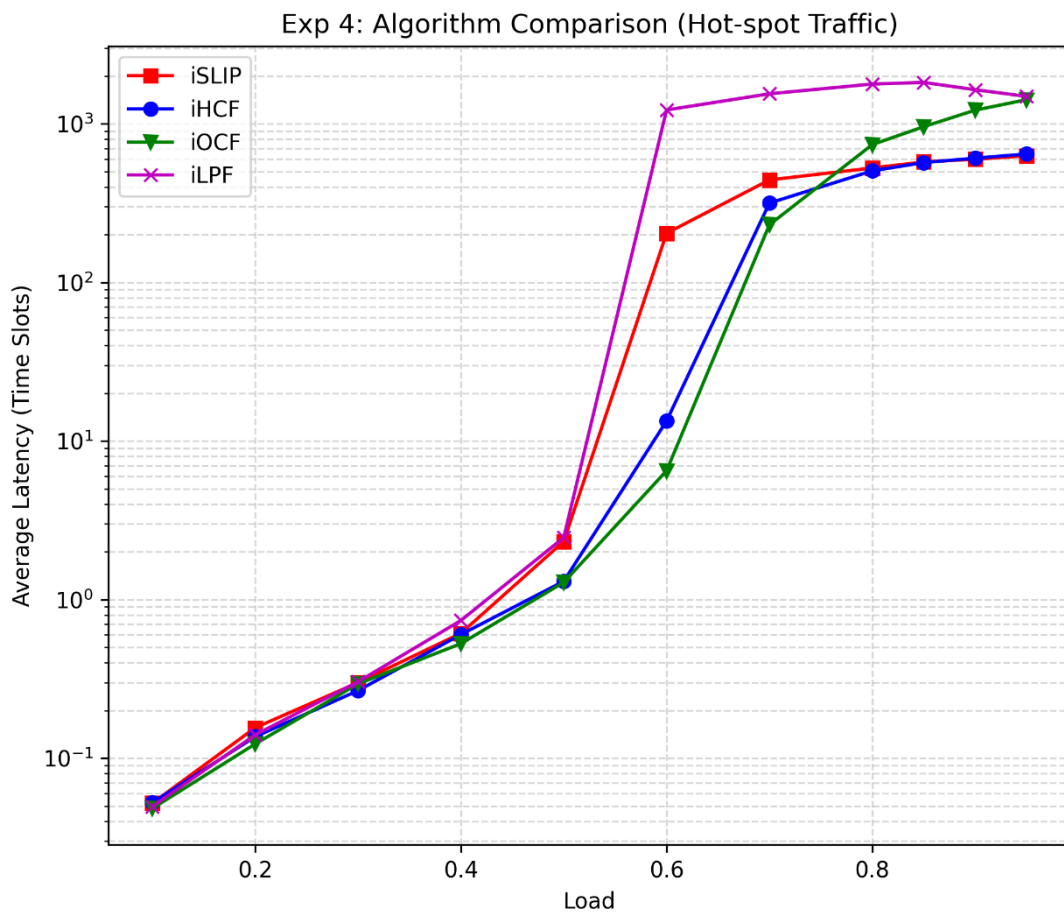


Figure 5 Performance of 4 algorithms under hot spot traffic

Honestly, the graph shows that all four algorithms struggle heavily at high loads. None of them can maintain low latency across the entire test.

iLPF fails very early. Its average latency spikes sharply at just a 0.5 load. iSLIP fails next. Its latency rises quickly after a 0.6 load.

iHCF and iOCF perform slightly better than the rest. They handle the congestion a bit longer. Their latencies spike later, around a 0.65 to 0.7 load. However, at very high loads (above 0.8), all algorithms suffer from extreme delays.

We believe this happens due to severe output contention combined with the single iteration limit.

The hot-spot port acts as a massive bottleneck. Many input ports constantly fight for this single output. Because our simulation only uses one iteration, the matching process is too weak. If an input loses the competition for the hot-spot, it has no time to match with a different, free output port. It simply wastes the time slot.

iHCF and iOCF delay the crash slightly. Their weight-based logic forces the switch to serve the oldest blocked queues fairly. However, the sheer volume of traffic targeting one single port eventually overwhelms the entire system. To fully resolve this severe bottleneck and avoid wasting slots, the scheduler would strictly require multiple iterations.

Conclusion

In this report, we evaluated the iHCF scheduling algorithm. We compared its performance against iSLIP, iOCF, and iLPF across various traffic models.

Our simulations confirm the core strengths of iHCF. It successfully combines the hardware simplicity of iSLIP with the smart weight-awareness of algorithms like iOCF.

Under uniform traffic, the standard iHCF performs perfectly. By using a small maximum counter limit, it forces frequent ties. It then uses round-robin pointers to break these ties. This entirely avoids the pointer synchronization that destroys pure weight-based algorithms. As a result, iHCF achieves 100% throughput and maintains low latency, perfectly matching iSLIP.

Under non-uniform and hot-spot traffic, iHCF shows its advantage over iSLIP. It uses simple saturating counters to approximate waiting times. This allows the algorithm to detect and prioritize heavily congested queues. Because of this, it adapts to unbalanced loads much better than weight-blind algorithms.

Although running only a single iteration limits the maximum performance under severe hot-spot bottlenecks, iHCF remains highly effective overall. It provides a practical, efficient, and hardware-friendly solution for modern high-speed switches.